Stackademic

AI + LLM + AWS + Cloud Computing + Linux +

RAG Is Dead. LLM Wiki — Andrej Karpathy’s Idea — Is What Comes Next



Jahangir Follow 9 min read · May 8, 2026

765 15 9



Stop losing insights to chat history. Let an AI build and maintain a living wiki from your documents.

Andrej Karpathy — the man who built Tesla's Autopilot AI from the ground up, who taught neural networks to hundreds of thousands through his legendary Stanford lectures, and who left OpenAI to go back to doing what

he loves most: thinking deeply about where AI is actually going — recently proposed something quietly radical.

He called it **LLM Wiki.**

And if you have been paying attention to how AI tools actually fail people in practice, it lands like a slap.

Here is the failure: every conversation you have with an AI starts from zero. You paste context, explain background, re-summarize what you already know — and once the tab closes, it is gone. The AI forgets everything. Your next session is blank again. Months of insights, research, and thinking: dissolved into chat history no one will ever read again.

Karpathy proposed a different model entirely. Instead of a chatbot that forgets, build a system where an LLM reads your documents and *writes* — maintaining a folder of interconnected Markdown files, one page per concept, linked together like Wikipedia. Knowledge does not disappear. It compounds.

This article is a practical guide to understanding what LLM Wiki is, the four major implementations available today, how to run any of them locally using

Ollama or vLLM, and which models to choose. All setup instructions and configs are also documented in this companion repository: github.com/jahangir842/llm-wiki-implementations

What Is LLM Wiki?

Traditional RAG (Retrieval-Augmented Generation) works like this: you ask a question, the system retrieves relevant chunks from your documents, and the LLM answers. The answer is good, but nothing is retained. The next query starts the same retrieval process from scratch.

LLM Wiki works differently. An LLM agent reads your sources — PDFs, notes, transcripts, web pages — and *writes* structured Markdown files from them. Each file covers one entity or concept. Pages link to each other using `[[wiki-links]]`. Over time, as you add more sources, the agent updates and extends existing pages rather than generating isolated answers.

The result is a living, browsable knowledge base that grows with you.

Think of it as the difference between asking a librarian a question and having the librarian actually reorganize the library for you, building a card catalog that gets better every time you bring in a new book.

The Four Implementations

The concept has been picked up by several developers, each taking a different angle. Here is a breakdown of the four most notable implementations.

1. nashsu/llm_wiki — The Desktop App

GitHub: github.com/nashsu/llm_wiki

This is the most visual implementation. Built with Tauri and React 19, it is a cross-platform desktop application with a three-column layout: Knowledge Tree on the left, Chat in the center, and a live Preview panel on the right. A separate Knowledge Graph view shows how your concepts connect, using community detection and relevance scoring to surface relationships.

What makes it stand out:

- Supports PDF, DOCX, PPTX, images, video, and web URLs as sources
- Two-step ingest: first it analyzes your source, then generates wiki content with every claim traced back to the original document
- Built-in Lint tool for ongoing maintenance — finds broken links, stale pages, and gaps
- Works with OpenAI, Anthropic, Google, Ollama, or any custom OpenAI-compatible endpoint
- Chrome extension for clipping web pages directly into your wiki

Best for: People who want a GUI, a knowledge graph, and support for multiple document types.

How to get started:

The easiest path is a pre-built binary. Download the `.deb`, `.AppImage`, `.dmg`, or `.msi` from the Releases page for your OS and install it directly.

To build from source:

```
git clone https://github.com/nashsu/llm_wiki.git
cd llm_wiki
npm install
npm run tauri dev
```

Once running, open Settings and point it at your LLM provider. If you are running Ollama locally, set the endpoint to `http://localhost:11434`. For vLLM, use `http://localhost:8000/v1`.

2. nvk/llm-wiki — The Agent Plugin

GitHub: github.com/nvk/llm-wiki

This one takes a completely different approach. Instead of a standalone app, it is a plugin for existing agent tools — Claude Code, OpenAI Codex, or any LLM agent via a universal `AGENTS.md` file.

The signature feature is **parallel research:** it spins up 5 to 10 concurrent agents that investigate a topic from different angles simultaneously, then compiles the findings into cross-referenced wiki articles with thesis-driven evidence evaluation (arguments for and against each claim).

It has zero runtime dependencies. It uses only the host agent's built-in file read/write and web fetch capabilities.

What makes it stand out:

- Parallel multi-agent research in a single command
- No extra API keys if you already use Claude Code — it reuses your subscription
- Generates not just wiki pages but also reports, slides, and study guides
- Universal `AGENTS.md` mode works with any LLM, including local models

Best for: Power users already inside Claude Code or Codex who want to research and document topics quickly.

How to get started:

```
# For Claude Code users
claude plugin install wiki@llm-wiki
```

```
# For Codex users
codex plugin marketplace add nvk/llm-wiki

# For any LLM agent (including local)
git clone https://github.com/nvk/llm-wiki.git
cp llm-wiki/AGENTS.md ./AGENTS.md
# Pass AGENTS.md as the system prompt to your local agent runner
```

Then:

```
/wiki init                                # Create ~/wiki/
/wiki:research "machine learning" --sources 10
@wiki query "what is attention mechanism"
@wiki audit                               # Find gaps
```

3. Pratiyush/llm-wiki — The Transcript Wiki

GitHub: github.com/Pratiyush/llm-wiki

This one solves a specific but very real problem: all those conversations you have had with Claude Code, Cursor, Gemini CLI, Codex, and Copilot are sitting in `.jsonl` files on your machine, and none of it is searchable or organized.

Pratiyush's tool reads those transcript files and builds a static HTML knowledge base from them. No LLM is needed at runtime — it processes the transcripts you already have and generates a browsable offline site.

What makes it stand out:

- Works offline after the build step — no API key needed to browse
- Page lifecycle management: draft → verified → stale → archived
- Auto-redacts API keys and usernames from your transcripts before publishing
- MCP server with 12 tools so other agents can query your wiki
- Exports in `llms.txt`, JSON-LD, and RSS formats for AI consumption

- Supports Claude Code, Cursor, Codex CLI, Gemini CLI, Copilot, and Obsidian

Best for: Anyone who wants to mine their existing LLM conversation history without running another LLM.

How to get started:

```
git clone https://github.com/Pratiyush/llm-wiki.git
cd llm-wiki
./setup.sh            # or setup.bat on Windows
pip install -e .
```

```
llmwiki sync          # Parse your transcripts
llmwiki build         # Generate static HTML
llmwiki serve         # Browse at http://localhost:8080
```

To expose the wiki to other agents via MCP:

```
llmwiki mcp start
```

4. lucasastorian/llmwiki — The MCP-Powered Wiki

GitHub: github.com/lucasastorian/llmwiki

This implementation focuses on one frustration: personal wikis are hard to maintain. You create them, they go stale, links break, summaries become outdated. The maintenance burden kills the habit.

The solution here is to let Claude do the maintenance via MCP. Point the tool at a directory, and Claude indexes everything, generates wiki pages, and keeps them updated as your files change. Your filesystem stays the source of truth — the wiki is a generated layer on top, not a replacement.

What makes it stand out:

- Single command to open any folder as a wiki: `./llmwiki open ~/research`

- Auto-updates links, summaries, and citations when files change
- Local SQLite index — no cloud search service required
- Optional Mistral integration for better PDF and scanned document OCR
- Built with a Python API backend and a React web UI

Best for: Researchers and writers with large document collections who want Claude to handle the organizational work.

How to get started:

```
git clone https://github.com/lucasastorian/llmwiki.git
cd llmwiki
```

Medium Search Play Games Write

```
export ANTHROPIC_API_KEY=your_key_here
./llmwiki init
./llmwiki open ~/your-documents
```

Open `http://localhost:3000` to browse.

Which Model Should You Use?

This is the practical question. Cloud APIs are easy but cost money and send your documents to external servers. Running locally with Ollama or vLLM gives you full control.

Ollama vs vLLM

Both run models locally, but they are built for different use cases.

Ollama is optimized for simplicity. One command to pull a model, one command to run it. Great for single-user, single-request workflows. If you are just experimenting or prototyping, Ollama is the right starting point.

vLLM is optimized for throughput. It uses PagedAttention for efficient long-context processing and delivers roughly 3x better throughput with 6x lower latency under concurrent load. When an LLM Wiki tool is ingesting dozens of documents in parallel or running multiple agents simultaneously, vLLM handles this far better than Ollama.

Bottom line: Start with Ollama to experiment. Switch to vLLM when you want performance.

Model Recommendations

For LLM Wiki tasks — structured writing, long document analysis, maintaining consistent cross-references — context window size and instruction-following quality are the two most important factors.

Use CaseRecommended ModelContext WindowRunnerBest all-roundQwen2.5–14B-Instruct128KvLLM or OllamaBest reasoningLlama-3.1–70B-Instruct128KvLLMVery large docsQwen2.5–1M1MvLLMLow VRAM (8GB)Llama-3.1–8B128KOllamaMid VRAM (16GB)Qwen2.5–14B128KOllama

Qwen2.5–14B-Instruct is the recommended starting point for most people. It has a 128K context window, excellent structured writing ability, and runs on a single 16GB GPU. It is available on both Ollama and HuggingFace.

Llama-3.1–70B-Instruct produces better reasoning and more nuanced summaries but requires around 48GB of VRAM. Use this if you have the hardware and care about output quality over speed.

Qwen2.5–1M is the choice for feeding very large documents — full books, long research corpora. It supports a 1 million token context window but requires vLLM with chunked prefill enabled.

Running with Ollama

```
ollama pull qwen2.5:14b
```

```
# The API is now at http://localhost:11434
# OpenAI-compatible endpoint: http://localhost:11434/v1
```

In nashsu/llm_wiki Settings: set Provider to Ollama, endpoint to `http://localhost:11434`, model to `qwen2.5:14b`.

Running with vLLM

```
pip install vllm
```

```
vllm serve Qwen/Qwen2.5-14B-Instruct \
  --max-model-len 131072 \
  --host 0.0.0.0 \
  --port 8000
```

In nashsu/llm_wiki Settings: set Provider to Custom (OpenAI-compatible), endpoint to `http://localhost:8000/v1`, model to `Qwen/Qwen2.5-14B-Instruct`.

For the extreme long-context use case:

```
vllm serve Qwen/Qwen2.5-1M \
  --enable-chunked-prefill \
  --max-model-len 1000000
```

Choosing the Right Implementation

Here is a quick decision guide:

- **You want a GUI with a knowledge graph and support for PDFs, images, and video** → nashsu/llm_wiki
- **You already use Claude Code or Codex and want parallel research with no extra setup** → nvk/llm-wiki
- **You have months of chat transcripts in .jsonl files and want to mine them** → Pratiyush/llm-wiki
- **You have a folder of research documents and want Claude to auto-maintain a wiki from them** → lucasastorian/llmwiki

These are not mutually exclusive. A reasonable setup is to use nashsu/llm_wiki as your primary ingestion and browsing interface, with nvk/llm-wiki for deep research sessions inside Claude Code, and Pratiyush/llm-wiki running periodically to absorb your conversation history.

Why This Matters

The core insight of LLM Wiki is that AI should not just answer questions — it should build and maintain understanding on your behalf.

The chatbot paradigm treats every interaction as disposable. LLM Wiki treats every interaction as an investment. Each document you feed in, each research session you run, makes the knowledge base more complete. The wiki you have in six months is fundamentally richer than the one you have today — not because you manually wrote anything, but because the agent kept working.

This is closer to how a research team actually operates: not by answering queries in isolation, but by maintaining a shared body of organized, cross-referenced knowledge that everyone can build on.

The tools are young and rough in places. But the concept is sound, the implementations are usable today, and running them locally means your documents never leave your machine.

Getting Started Checklist

1. Pick your implementation based on the decision guide above
2. If using local models: install Ollama (`curl -fsSL https://ollama.com/install.sh | sh`) or vLLM (`pip install vllm`)

3. Pull `qwen2.5:14b` as your starting model — it runs on consumer hardware and handles the task well
4. Start small: feed in 5–10 focused documents on one topic before scaling up
5. Explore the knowledge graph after the first ingest — the connections it surfaces are often surprising

The best time to start building your wiki was six months ago. The second best time is now.

All four implementations are open source and actively maintained. Full setup guides, configs, and model recommendations are collected in the companion repository: github.com/jahangir842/llm-wiki-implementations

A message from our Founder

Hey, Sunil here. I wanted to take a moment to thank you for reading until the end and for being a part of this community. Did you know that our team run

these publications as a volunteer effort to over 3.5m monthly readers? We don't receive any funding, we do this to support the community.

If you want to show some love, please take a moment to follow me on LinkedIn, TikTok, Instagram. You can also subscribe to our weekly newsletter. And before you go, don't forget to clap and follow the writer!

AI + LLM + AWS + Cloud Computing + Linux +



Written by Jahangir

261 followers · 45 following

Follow

Cloud | DevOps Engineer | AWS & Azure Solution Architect | VMware EsXi | Linux | DevOps | Git | Dockers | kubernetes | Python | Bash Scripting



Published in Stackademic

88K followers · Last published 4 hours ago

Follow

Stackademic is a learning hub for programmers, devs, coders, and engineers. Our goal is to democratize free coding education for the world.

Responses (15)

Krzysztof

What are your thoughts?

Remsy Schmilinsky he/him
May 13

Thanks for sharing, however RAG ain't dead https://github.com/schmitech/orbit

8 1 reply Reply

Chris Perrin he/him
May 12

> The next query starts the same retrieval process from scratch.

Seems like you could build RAG into the server?

8 1 reply Reply



Abdullah zahid
May 12

The "slap" metaphor is spot on it’s wild how we’ve accepted "amnesia by design" as the standard for AI interaction. Transitioning from disposable chats to compounding knowledge is the logical next step for anyone doing serious research.

The breakdown... more

13 1 reply Reply

See all responses

More from Jahangir and Stackademic



Jahangir · Jun 3

The Beginner's Guide to Learning Agentic AI: From Zero to Your Firs...

A comprehensive roadmap for anyone starting their journey into autonomous AI...

212 9 2



In Stackademic by Umesh Kumar Yadav · Sep 18

JDK 27 has been officially released, I'm stunned!

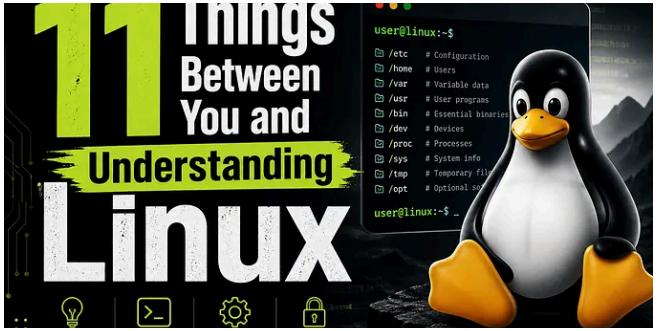
On September 15, 2026, Oracle officially released JDK 27, corresponding to JSR 402,...

292 9 3



In Stackademic by Jerry PM · Sep 6

9 Cool Repositories I Found on the Internet This Week



In Dev Genius by Jahangir · Apr 27

You're Not Bad at Linux, You're Missing These 11 Concepts

The SKILL.md quietly became the unit of distribution and Google shipping one is the...

662 14 3

There’s a moment every Linux user remembers.

429 6 1

See all from Jahangir

See all from Stackademic

Help Status About Careers Press Blog Store Privacy Rules Terms Text to speech